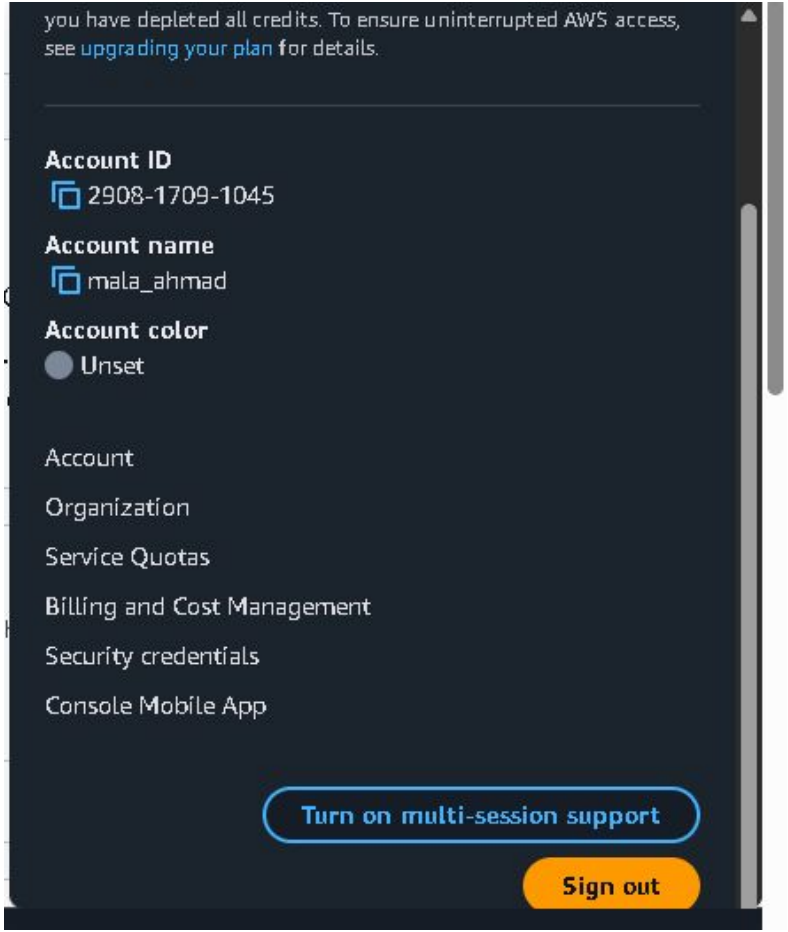
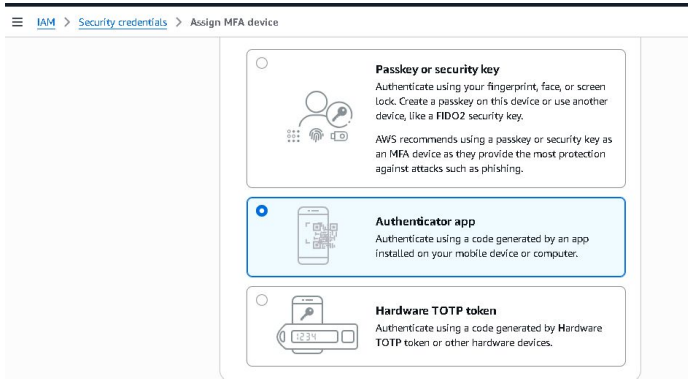
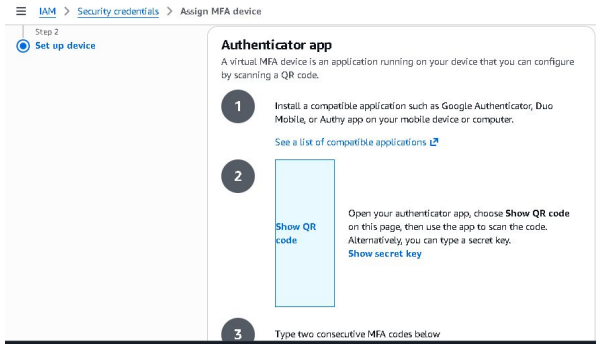
Task 1: AWS IAM Security Enhancement

Accessing Security Credentials

Log in to the AWS Management Console as the root user

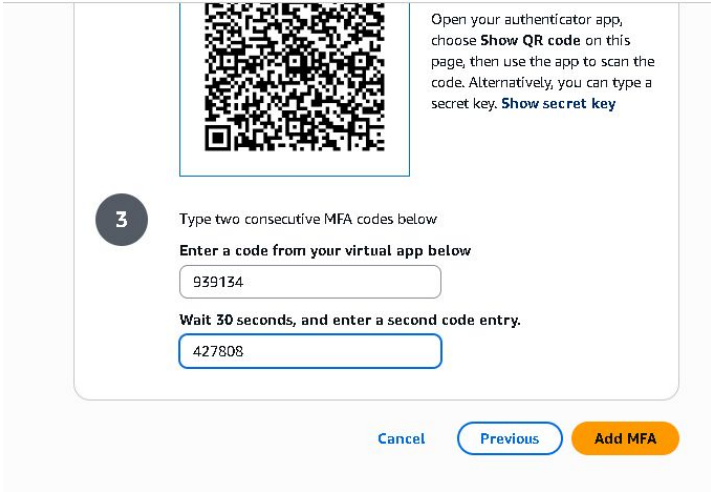


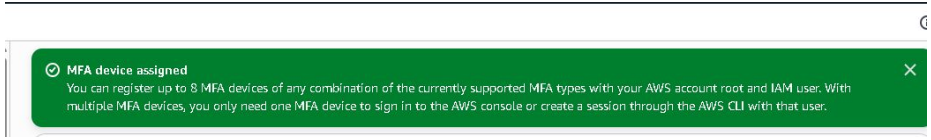




Assigning MFA

Click the **"Assign MFA device"** button within the "Multi-factor authentication (MFA)" section.



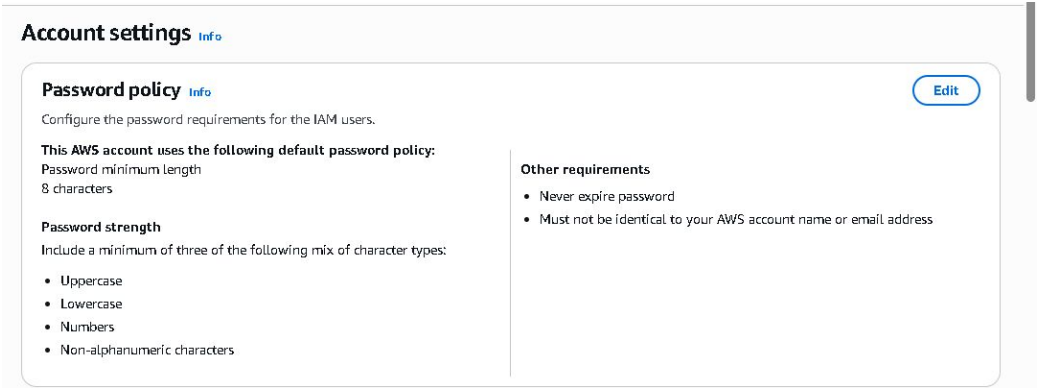


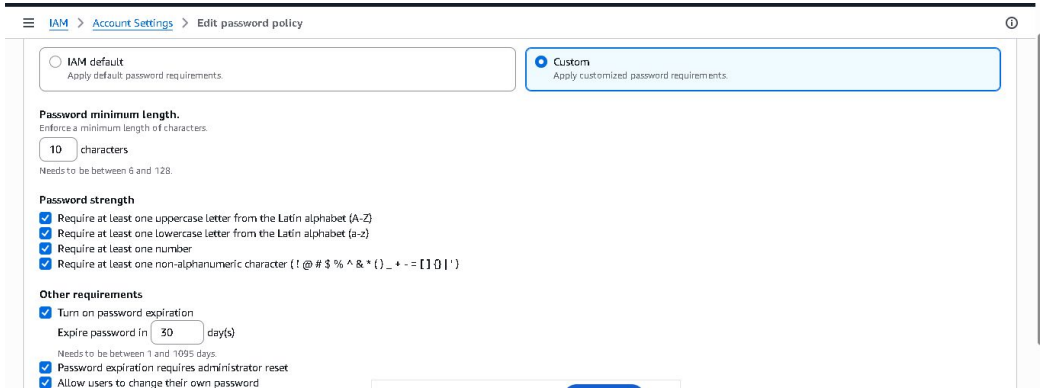
Initializing the Virtual MFA Device

Use the mobile app to scan the displayed QR code to link the AWS account to the virtual device

Reviewing the Default Password Policy

- Navigate to the IAM Dashboard and select "Account settings"
- Review the current "IAM default" policy
- Click "Edit" to establish a custom policy tailored to company standards
- Select the "Custom" button to define specific security requirements
- Set "Password minimum length" to 10 characters
- Enforce Strength by checking boxes for Uppercase, Lowercase, Numbers, and Non-alphanumeric characters
- Turn on "Password expiration" and set it to 30 days
- Enable "Password expiration requires administrator reset" and allow users to change their own passwords

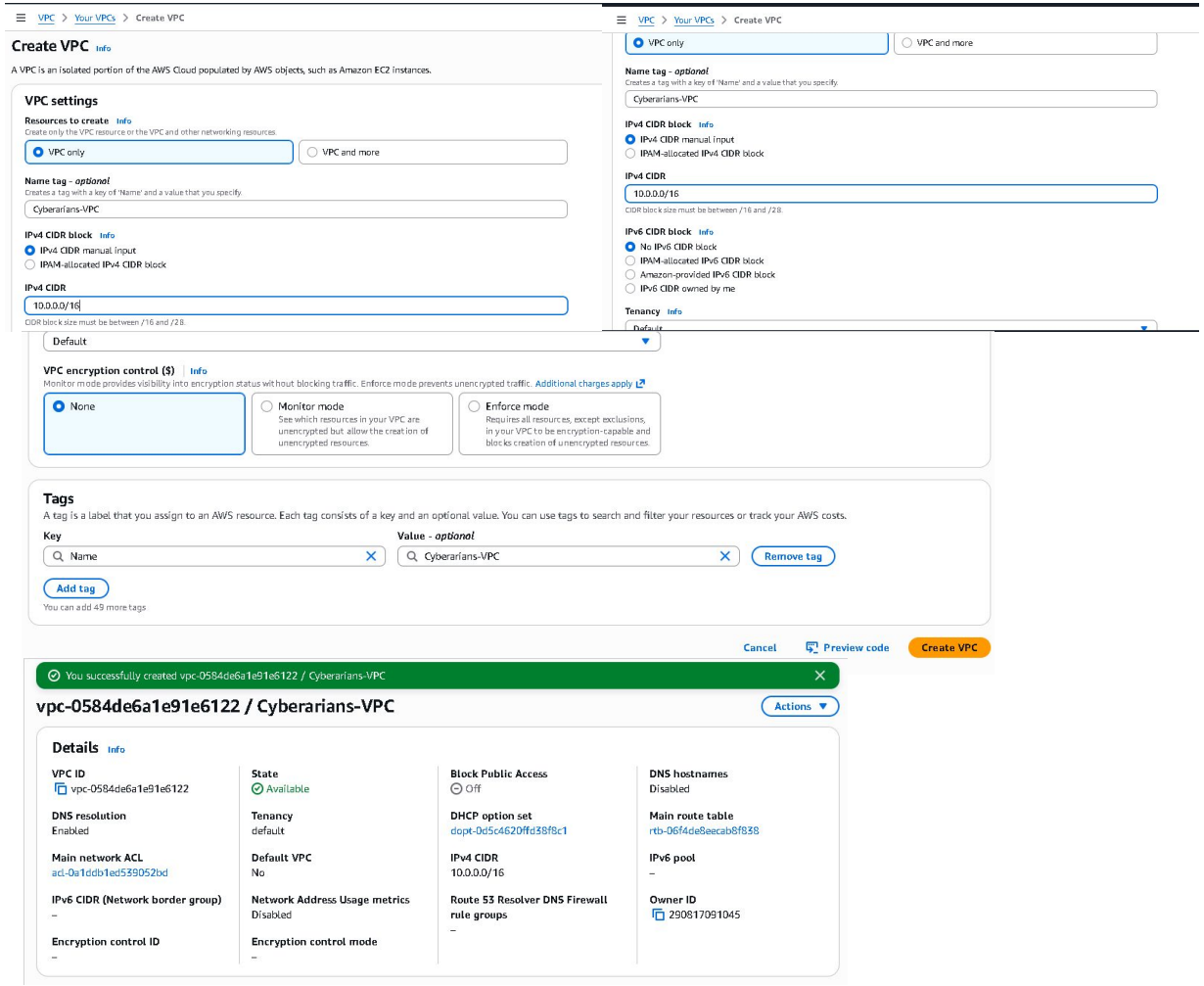




Task 2: VPC and Subnet Foundation

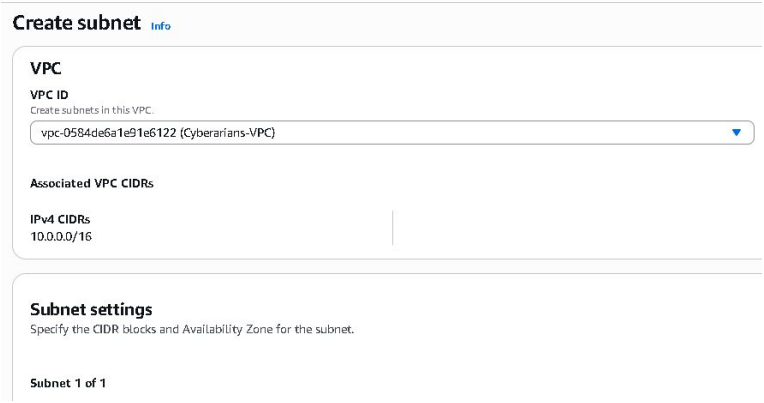
CREATING CUSTOM VPC

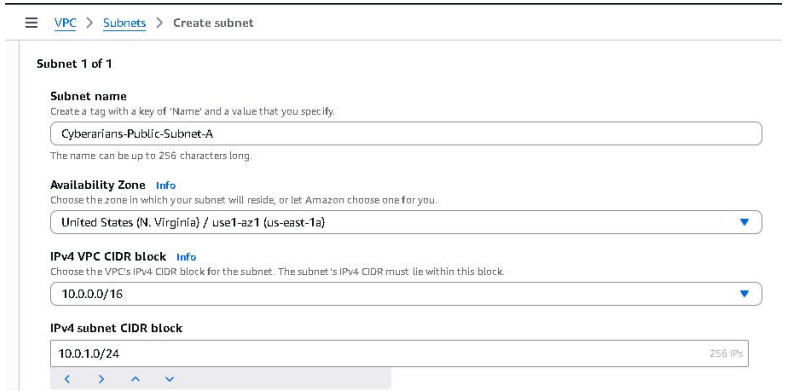
- Initiate the creation of a new VPC named **"Cyberarians-VPC"**.
- Define the network range using a **CIDR block of 10.0.0.0/16** to provide ample IP address space for current and future resources.

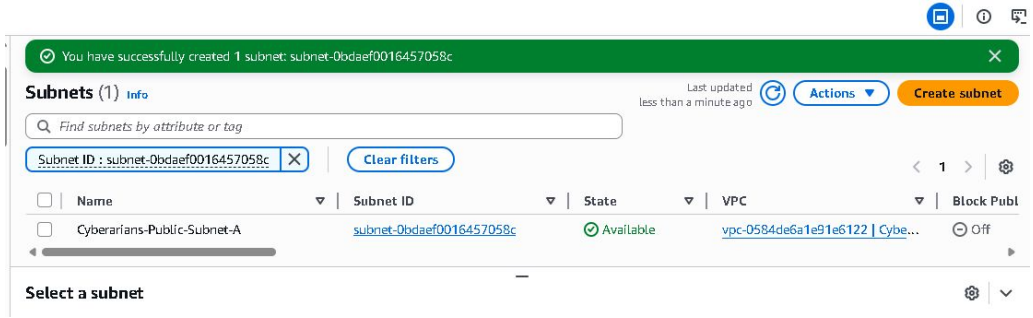


Provisioning Public Subnets

- Create a subnet named **"Cyberarians-Public-Subnet-A"** within the new VPC.
- Assign a specific CIDR block of **10.0.1.0/24** and place it in the us-east-1a Availability Zone.

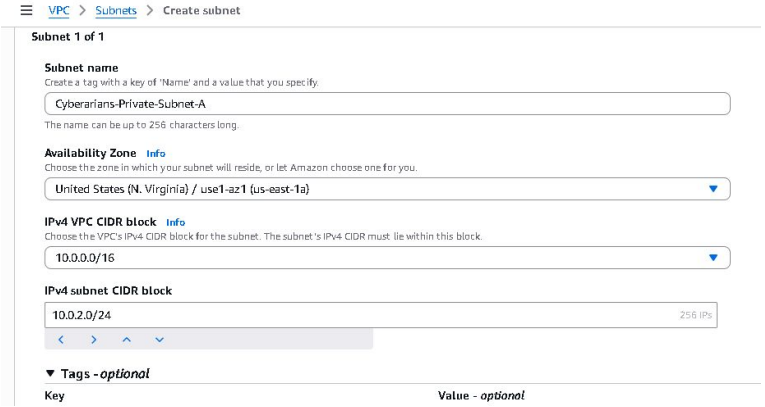


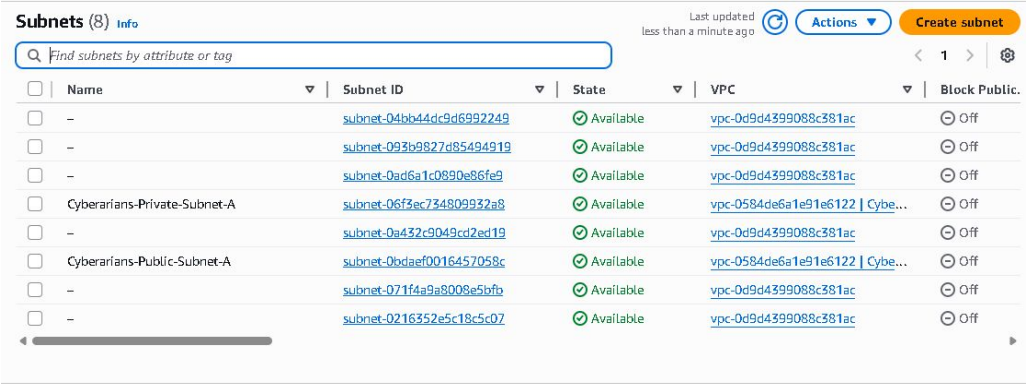




Provisioning Private Subnets

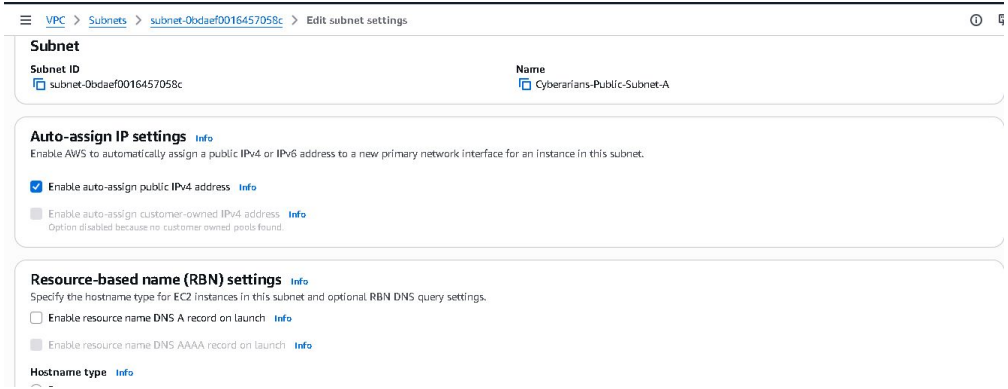
- Create a secondary subnet named **"Cyberarians-Private-Subnet-A"**.
- Assign the CIDR block **10.0.2.0/24**, ensuring logical separation from the public network.





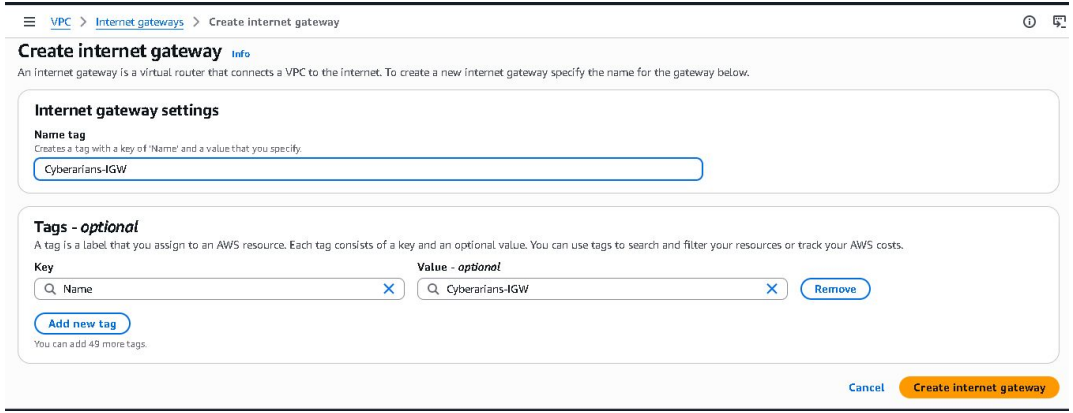
Enabling Public Connectivity

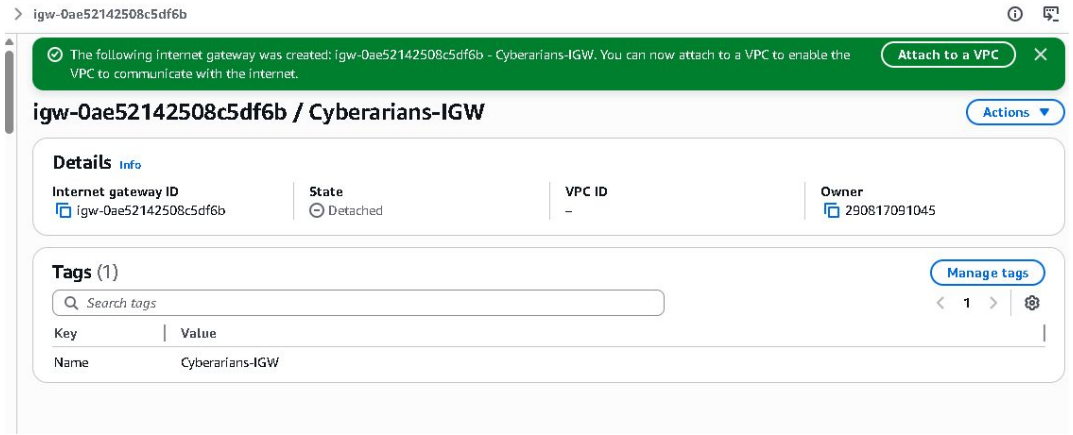
- Modify the settings for the public subnet to **"Enable auto-assign public IPv4 address"**.
- This ensures that any EC2 instance launched here automatically receives a public IP for internet accessibility.



Deploying the Internet Gateway

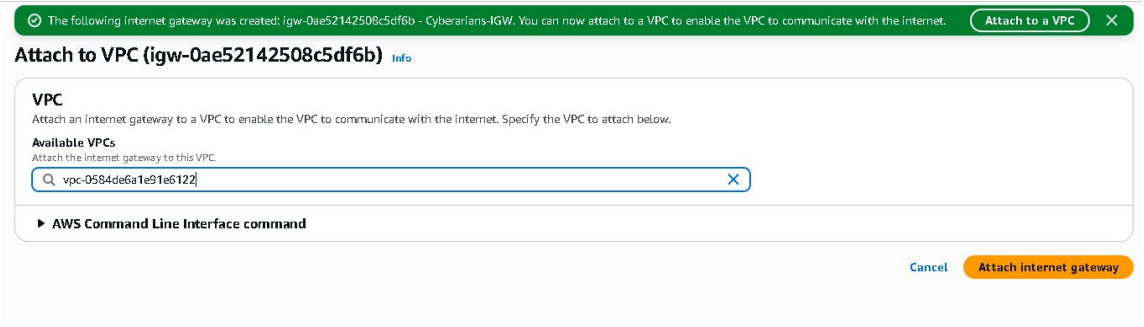
- Create an Internet Gateway named **"Cyberarians-IGW"** to serve as the bridge between the VPC and the public internet.

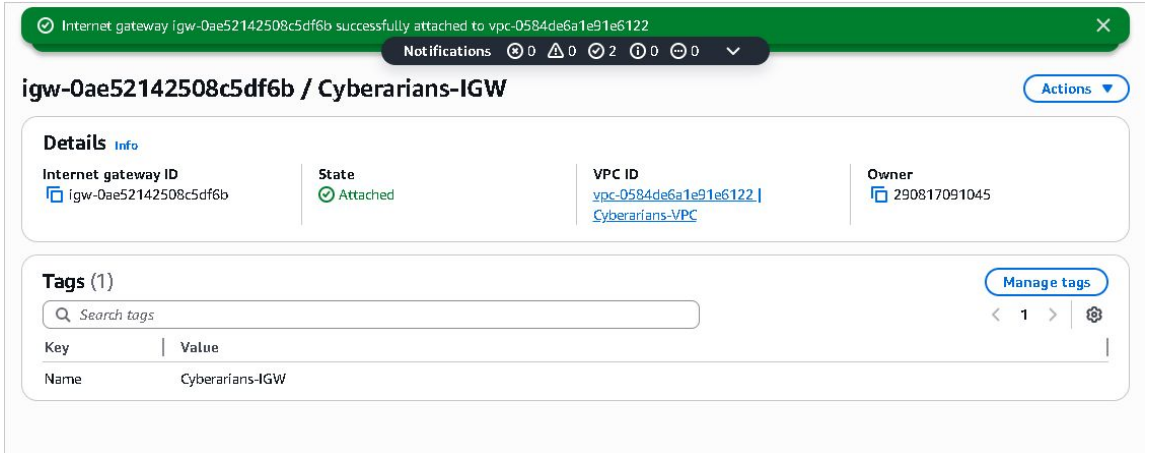




Attaching the IGW to the VPC

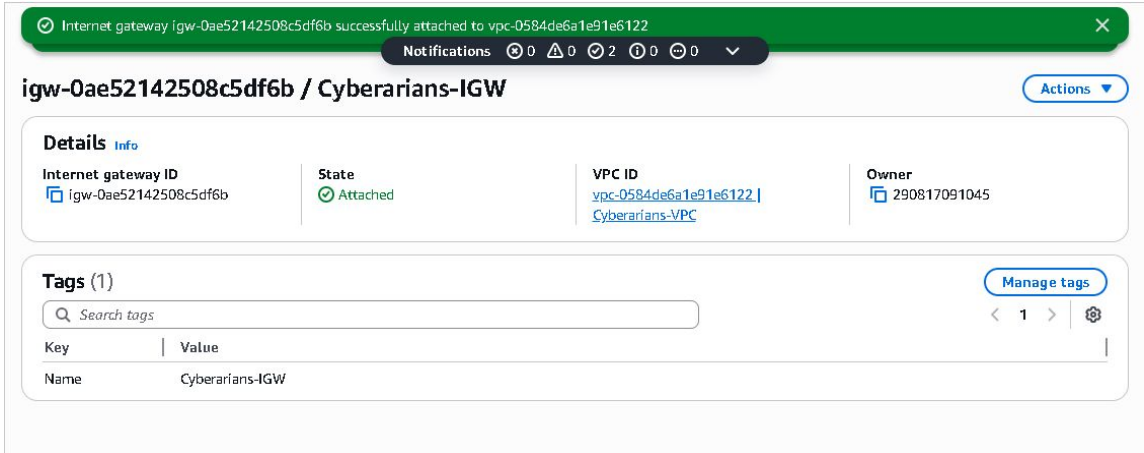
- Explicitly attach the newly created gateway to **"Cyberarians-VPC"** to enable external traffic flow.





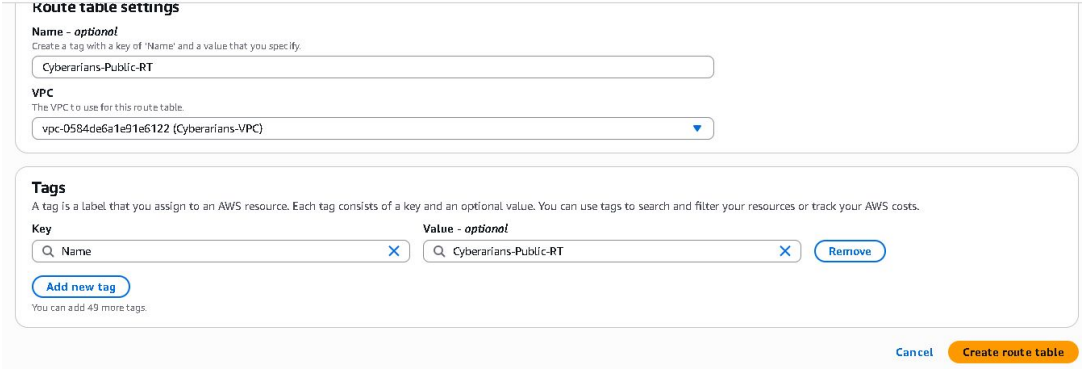
Attaching the IGW to the VPC

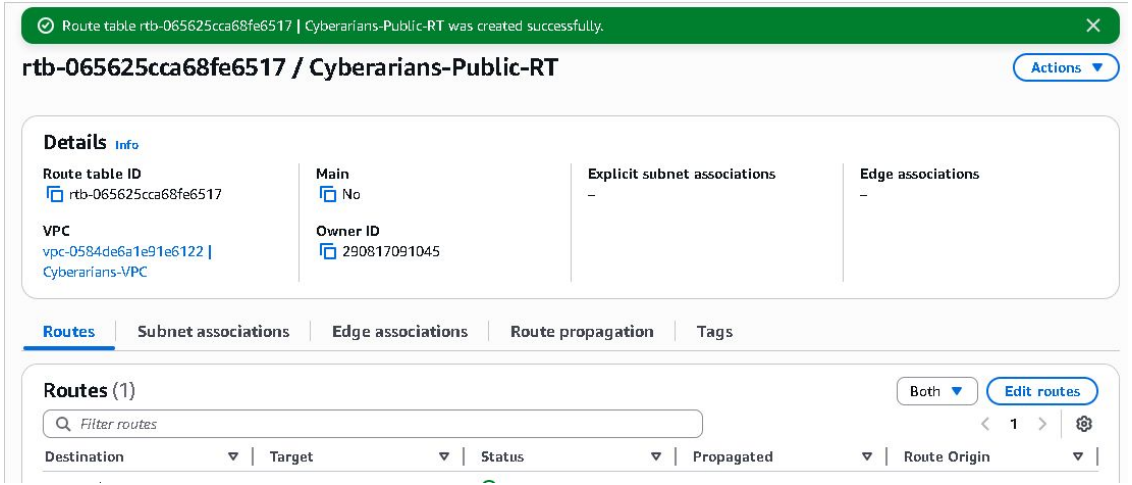
- Explicitly attach the newly created gateway to **"Cyberarians-VPC"** to enable external traffic flow.

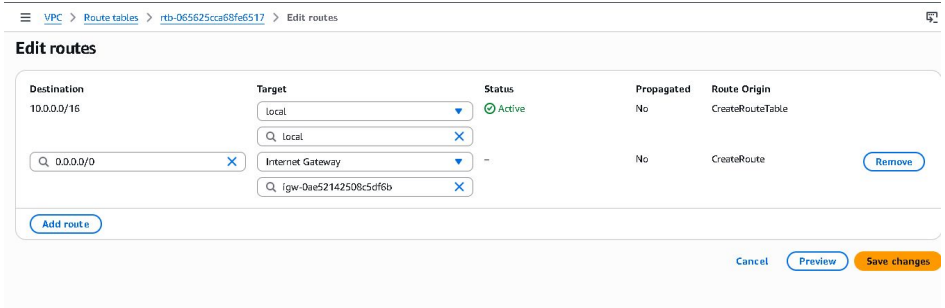


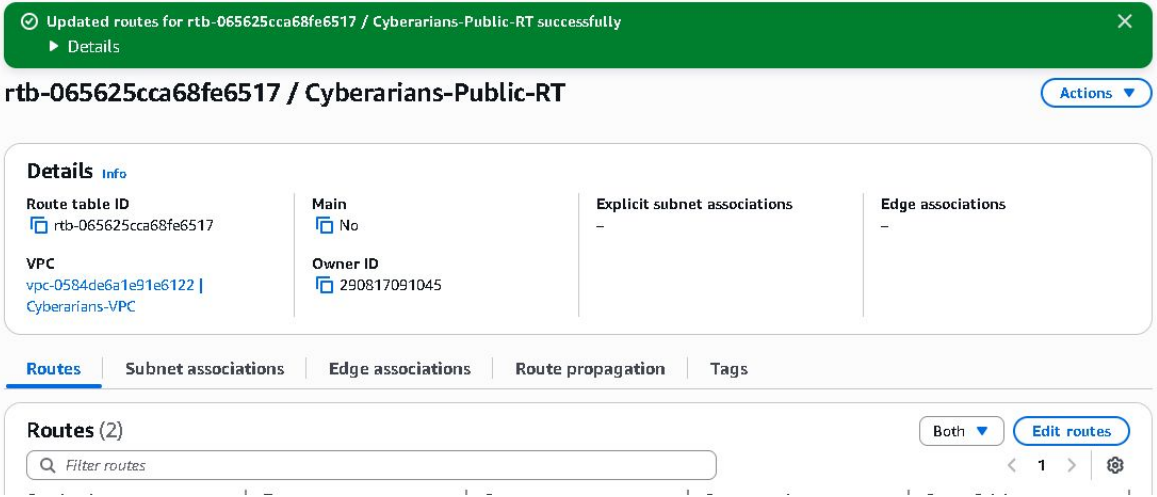
Configuring the Public Route Table

- Create a custom route table named **"Cyberarians-Public-RT"** specifically for public-facing resources.
- Add a **default route (0.0.0.0/0)** targeting the Internet Gateway (igw-...) to direct all internet-bound traffic.



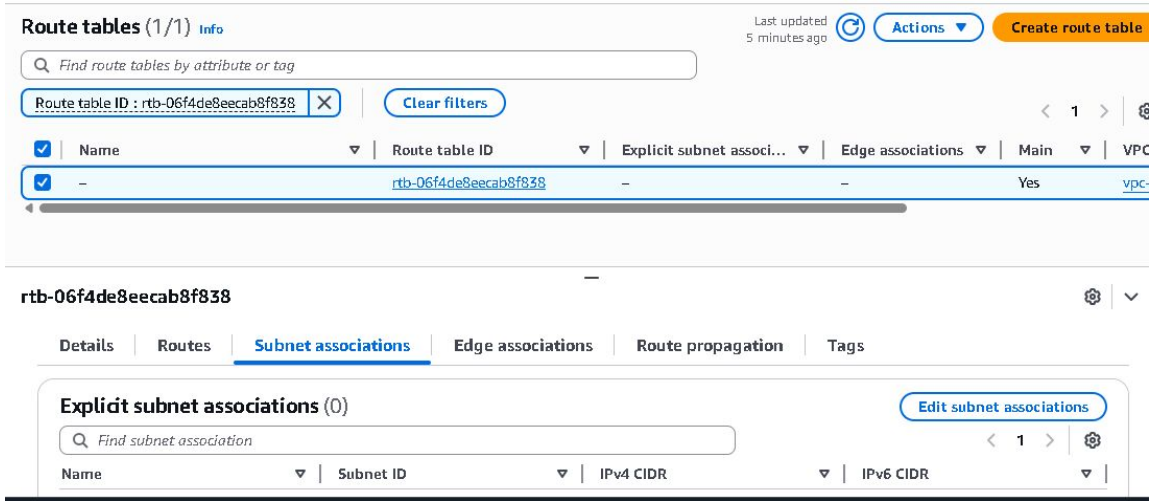


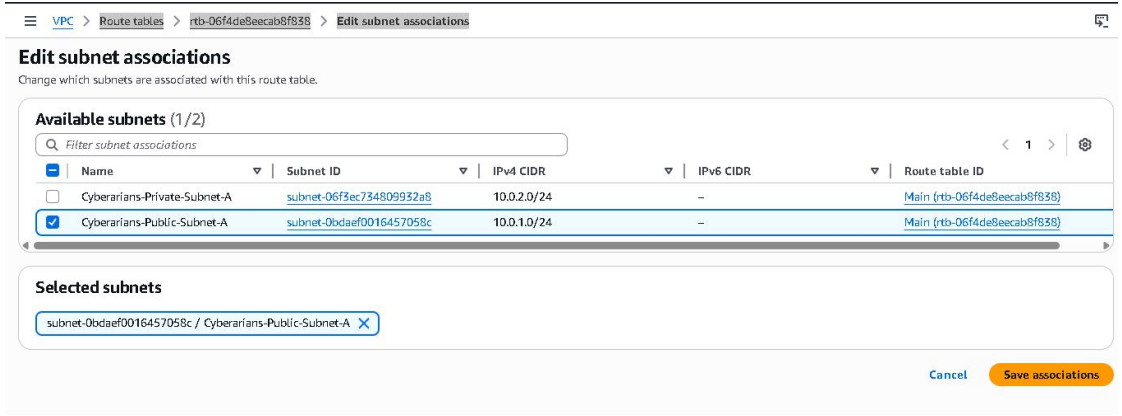

VPC > Route tables > rtb-065625cca68fe6517 > Edit routes
Edit routes
Destination
Target
Status
Propagated
Route Origin
10.0.0.0/16
local
Active
No
CreateRouteTable
local
0.0.0.0/0
Internet Gateway
–
No
CreateRoute
Remove
igw-0ae52142508c5df6b
Add route
Cancel
Preview
Save changes


Updated routes for rtb-065625cca68fe6517 / Cyberarians-Public-RT successfully
Details
rtb-065625cca68fe6517 / Cyberarians-Public-RT
Actions
Details Info
Route table ID
rtb-065625cca68fe6517
Main
No
Explicit subnet associations
–
Edge associations
–
VPC
vpc-0584de6a1e91e6122 | Cyberarians-VPC
Owner ID
290817091045
Routes
Subnet associations
Edge associations
Route propagation
Tags
Routes (2)
Both
Edit routes
Filter routes
1

Finalizing Subnet Associations

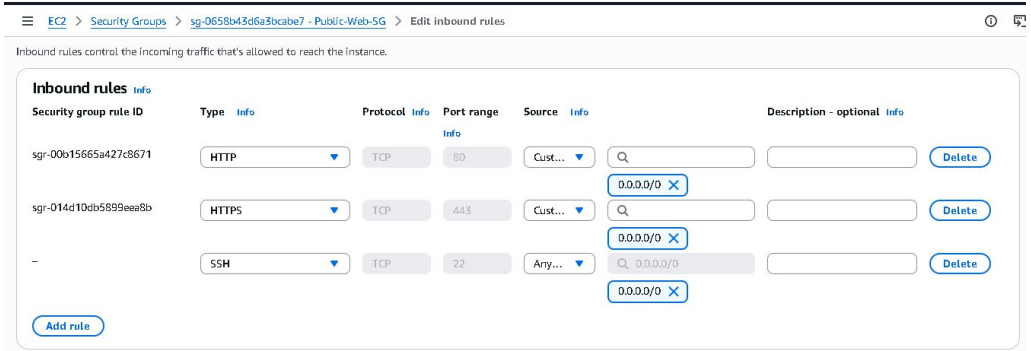
- Associate **"Cyberarians-Public-Subnet-A"** with the Public Route Table.
- The private subnet remains associated with the **Main Route Table** (which lacks an internet route), keeping it securely isolated.

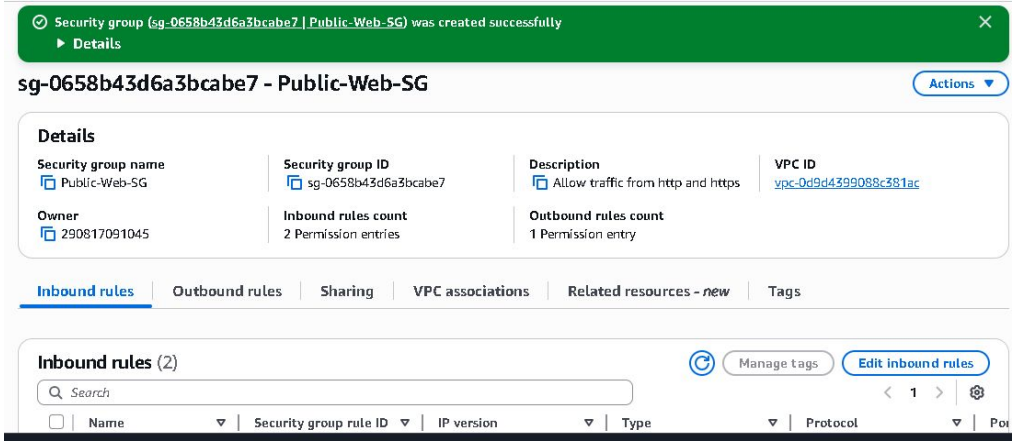




Defining the Public Security Group

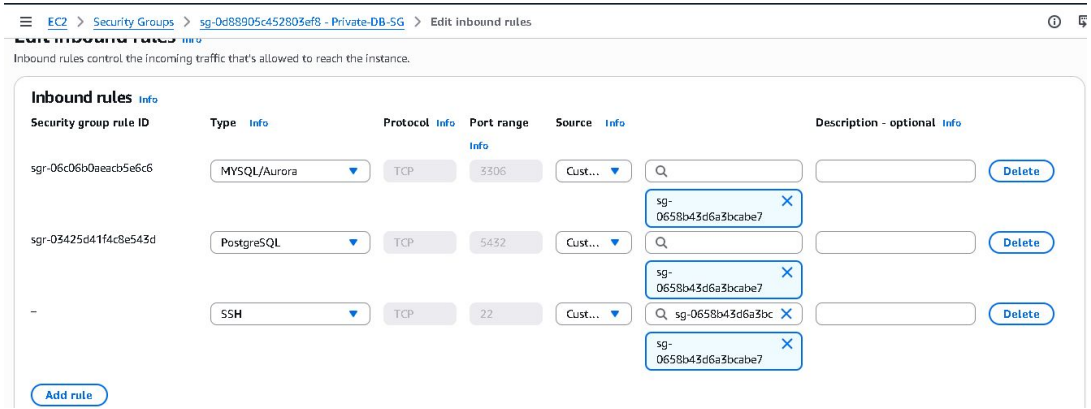
- Configure **"Public-Web-SG"** with inbound rules to allow **HTTP (80)**, **HTTPS (443)**, and **SSH (22)** traffic from anywhere (0.0.0.0/0).

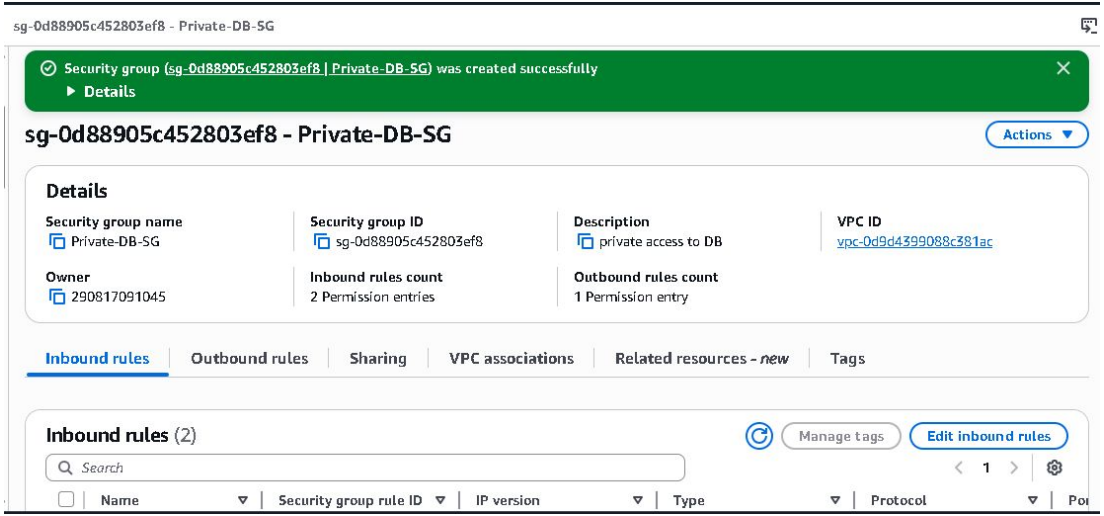




Defining the Private (Database) Security Group

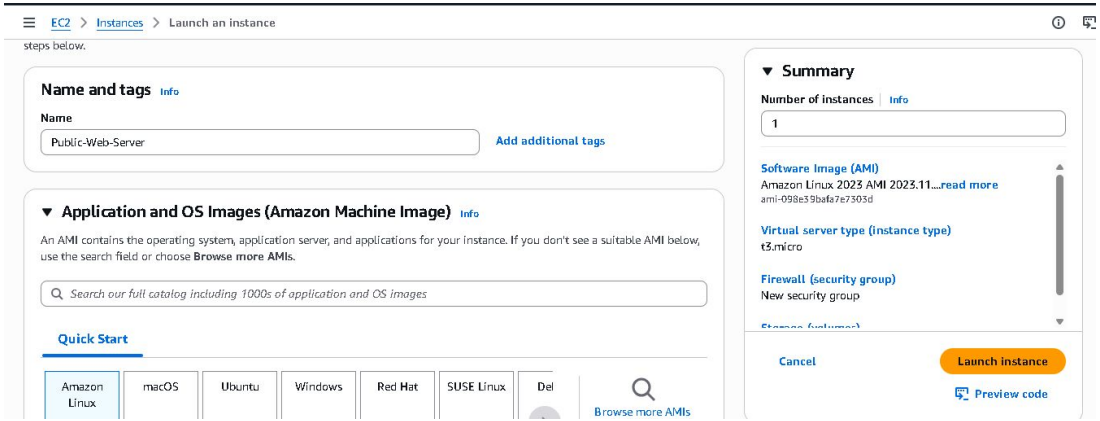
- Configure **"Private-DB-SG"** to restrict access to the database layer.
- Set inbound rules to allow **MySQL (3306)** or **PostgreSQL (5432)** traffic **only if it originates from the "Public-Web-SG"** security group ID.

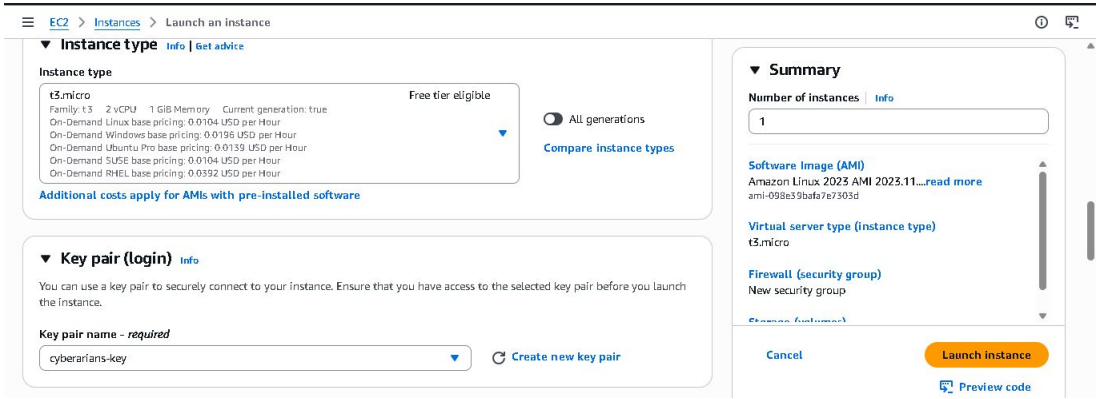




Launching EC2 Instances

- Launch an EC2 instance named **"Public-Web-Server"** in the public subnet and a private instance for testing connectivity.





EC2 > Instances > Launch an instance

Network settings Info

VPC - *required* Info

vpc-0584de6a1e91e6122 (Cyberarians-VPC)
10.0.0.0/16

Subnet Info

subnet-06f3ec734809932a8 Cyberarians-Private-Subnet-A
VPC: vpc-0584de6a1e91e6122 Owner: 290817091045
Availability Zone: us-east-1a (use1-az1) Zone type: Availability Zone
IP addresses available: 250 CIDR: 10.0.2.0/24)

Create new subnet

Auto-assign public IP Info

Enable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

- Create security group
- Select existing security group

Common security groups Info

Summary

Number of instances Info

1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.11....read more
ami-098e39bafa7e7303d

Virtual server type (instance type)
t3.micro

Firewall (security group)

Cancel | Launch instance | Preview code

aws | Search [Alt+S] | United States (N. Virginia) | mala_ahmad (2908-1709-1045) | mala_ahmad

EC2 > Instances > Launch an instance

Success
Successfully initiated launch of instance (i-080e64560b83ee4c5)

EC2 > Instances > Launch an instance

VPC - *required* Info

vpc-0584de6a1e91e6122 (Cyberarians-VPC)
10.0.0.0/16

Subnet Info

subnet-06f3ec734809932a8 Cyberarians-Private-Subnet-A
VPC: vpc-0584de6a1e91e6122 Owner: 290817091045
Availability Zone: us-east-1a (use1-az1) Zone type: Availability Zone
IP addresses available: 250 CIDR: 10.0.2.0/24)

Create new subnet

Auto-assign public IP Info

Disable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

- Create security group
- Select existing security group

Common security groups Info

Select security groups

Compare security group rules

Private-Web-SG-2 sg-0b2ea72e7b289ccd2
VPC: vpc-0584de6a1e91e6122

Summary

Number of instances Info

1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.11....read more
ami-098e39bafa7e7303d

Virtual server type (instance type)
t3.micro

Firewall (security group)
Private-Web-SG-2

Cancel | Launch instance | Preview code

Public Internet Access Test

- Access the **"Public-Web-Server"** via SSH and run a ping google.com command.
- Success confirms the instance has a public IP, is correctly routed through the IGW, and the security group allows outbound traffic.

```
C:\Users\Mala\Downloads>ssh -i "Public-Web-Server-key.pem" ec2-user@32.195.69.34
The authenticity of host '32.195.69.34 (32.195.69.34)' can't be established.
ED25519 key fingerprint is SHA256:jfZT/Yy8Hw6QyjhBLxO/Td9wzoKnLbHQXUXJ9Y0Bg8c.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '32.195.69.34' (ED25519) to the list of known hosts.
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Thu Apr 16 20:49:54 2026 from 18.206.107.27
[ec2-user@ip-10-0-1-152 ~]$
[ec2-user@ip-10-0-1-152 ~]$ ping google.com
PING google.com (172.253.62.100) 56(84) bytes of data.
64 bytes from bc-in-f100.1e100.net (172.253.62.100): icmp_seq=1 ttl=102 time=1.93 ms
64 bytes from bc-in-f100.1e100.net (172.253.62.100): icmp_seq=2 ttl=102 time=1.92 ms
64 bytes from bc-in-f100.1e100.net (172.253.62.100): icmp_seq=3 ttl=102 time=1.91 ms
64 bytes from bc-in-f100.1e100.net (172.253.62.100): icmp_seq=4 ttl=102 time=1.93 ms
64 bytes from bc-in-f100.1e100.net (172.253.62.100): icmp_seq=5 ttl=102 time=2.09 ms
64 bytes from bc-in-f100.1e100.net (172.253.62.100): icmp_seq=6 ttl=102 time=1.95 ms
^C
--- google.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt min/avg/max/mdev = 1.914/1.954/2.089/0.061 ms
[ec2-user@ip-10-0-1-152 ~]$
```

i-0d682dbee6a37af0f (Public-web-server)

PublicIPs: 32.195.69.34 PrivateIPs: 10.0.1.152

Private Subnet Isolation Test

- SSH from the public server into the **private instance** (10.0.2.122) and attempt to ping the internet.
- **Failure (100% packet loss)** successfully verifies that the private subnet is isolated and cannot be reached by or reach the internet directly.

```
[ec2-user@ip-10-0-1-152 ~]$ chmod 400 private-key.pem
[ec2-user@ip-10-0-1-152 ~]$ ssh -i "private-key.pem" ec2-user@10.0.2.122
       #_
       ####_        Amazon Linux 2023
      _#####\
          \###|
            \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
             V~' '->
              /
            _/
          _/ _/
        _/m/'
[ec2-user@ip-10-0-2-122 ~]$ ping google.comping google.com
ping: google.comping: Name or service not known
[ec2-user@ip-10-0-2-122 ~]$ ping google.com
PING google.com (142.251.163.139) 56(84) bytes of data.

^C
--- google.com ping statistics ---
91 packets transmitted, 0 received, 100% packet loss, time 93637ms

[ec2-user@ip-10-0-2-122 ~]$
```

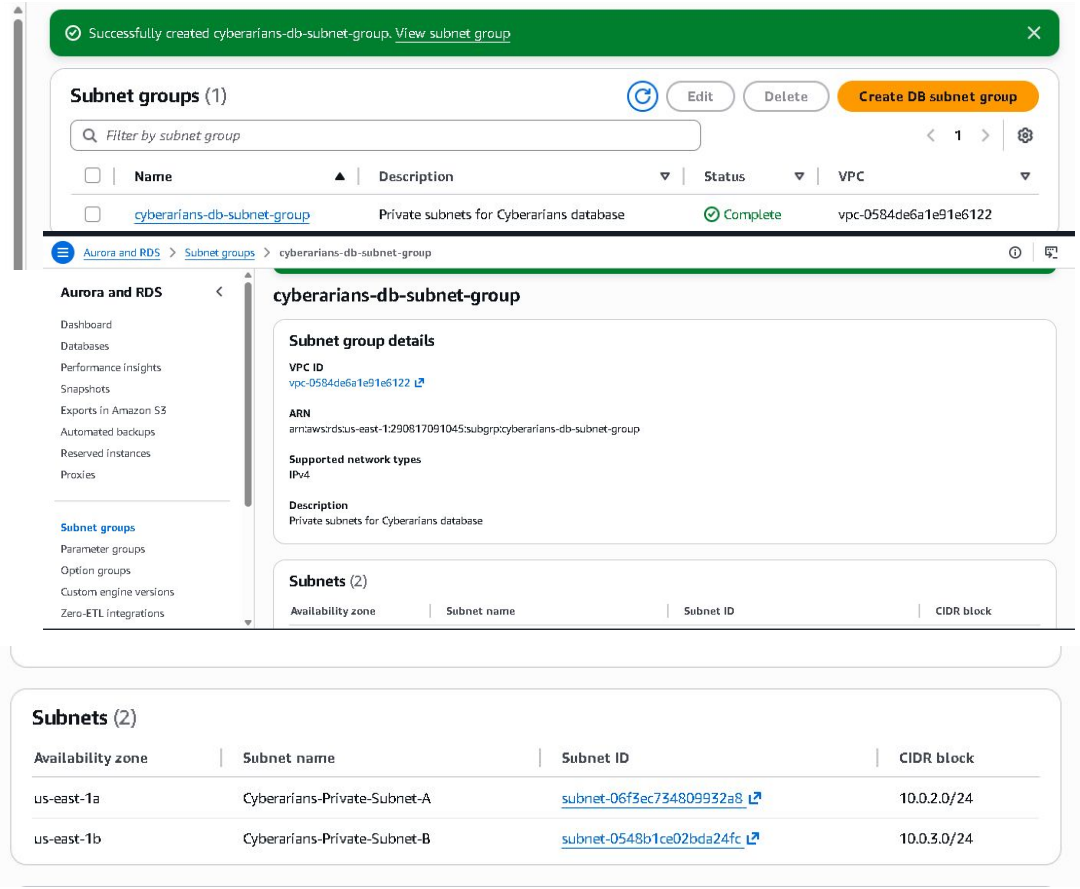
Task 3: Deploying a Secure RDS Backend

Establishing Multi-AZ Network Foundations

Created a second private subnet (Cyberarians-Private-Subnet-B).

Placed in a separate Availability Zone for high availability.

Ensured isolated routing (no internet access).

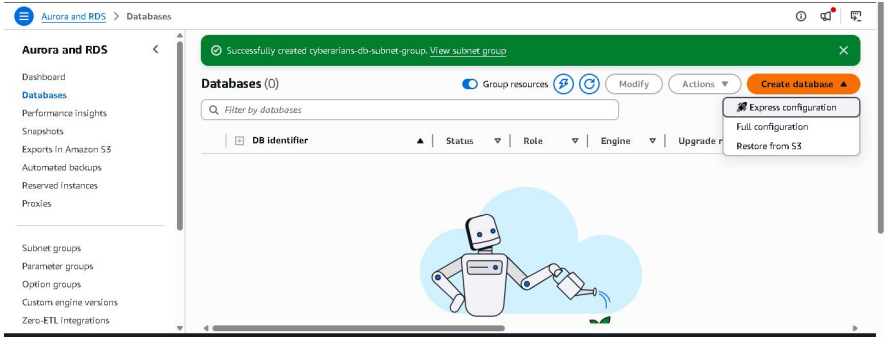


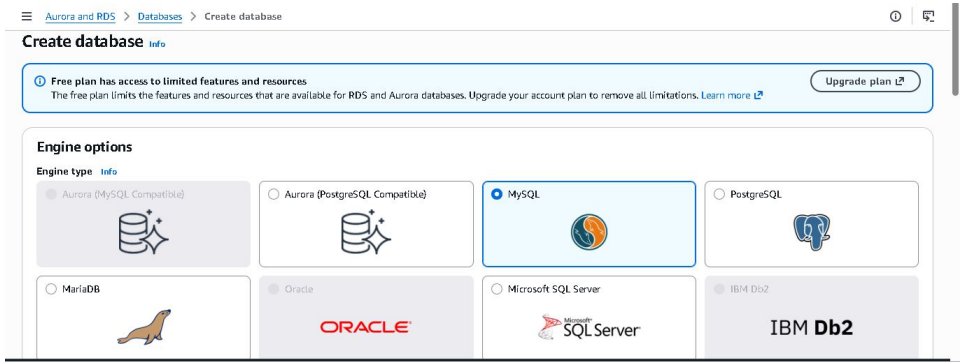
Database Configuration

Engine: Selected MySQL (Industry standard relational database).

Tier: Utilized the AWS Free Tier (Single-AZ deployment).

Credentials: Established secure Master Username and Password for backend access.





Aurora and RDS > Databases > Create database

Choose a database creation method

- (●) **Full configuration**
  You set all of the configuration options, including ones for availability, security, backups, and maintenance.
- ( ) **Easy create**
  Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Templates

Choose a sample template to meet your use case.

- ( ) **Production**
  Use defaults for high availability and fast, consistent performance.
- ( ) **Dev/Test**
  This instance is intended for development use outside of a production environment.
- (●) **Free tier**
  Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS.

Availability and durability

Deployment options Info

Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the Amazon RDS service level agreement (SLA).

Aurora and RDS > Databases > Create database

DB instance identifier Info

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

cyberarians-db

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username Info

Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

- ( ) Managed in AWS Secrets Manager - *most secure*
  RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.
- (●) Self managed
  Create your own password or have RDS create a password that you manage.

[ ] Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password Info

Aurora and RDS > Databases > Create database

Connectivity Info

Compute resource

Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

- (selected) **Don't connect to an EC2 compute resource**
  Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.
- **Connect to an EC2 compute resource**
  Set up a connection to an EC2 compute resource for this database.

Virtual private cloud (VPC) Info

Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Cyberarians-VPC (vpc-0584de6a1e91e6122)
3 Subnets, 2 Availability Zones

After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

After a database is created, you can't change its VPC.

DB subnet group Info

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

cyberarians-db-subnet-group
2 Subnets, 2 Availability Zones

Backup retention period Info

The number of days (1-35) for which automatic backups are kept.

0 days

Backup window Info

The daily time range (in UTC) during which RDS takes automated backups.

- (selected) Choose a window
- No preference

Start time

03:00 UTC

Duration

0.5 hours

Backup tags Info

Copies tags from the source database to the automated backup and snapshots respectively.

- [ ] Copy tags to automated backup
  This is a one-time setting. Future tag modifications need manual updates.
- [x] Copy tags to snapshots

Backup replication Info

- [ ] Enable replication in another AWS Region
  Enabling replication automatically creates backups of your DB instance in the selected Region, for disaster

Creating database cyberarians-db

Your database might take a few minutes to launch. You can use settings from cyberarians-db to simplify configuration of **suggested database add-ons** while we finish creating your DB for you.

View connection details

Databases (1)

Group resources | Modify | Actions | Create database

Filter by databases

1

DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...
cyberarians-db	Creating	Instance	MySQL Co...	SECOND	us-east-1a

Successfully created database cyberarians-db

You can use settings from cyberarians-db to simplify configuration of **suggested database add-ons** while we finish creating your DB for you.

View connection details

Databases (1)

Group resources | Modify | Actions | Create database

Filter by databases

1

DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...
cyberarians-db	Available	Instance	MySQL Co...	SECOND	us-east-1a

Task 4: Establishing a Secure Cloud Infrastructure for Cyberarians

Connect and Install the Web Stack

Update the server:

```
sudo dnf update -y
```

Install Apache, MariaDB, and PHP:

```
sudo dnf install -y httpd wget php-fpm
php-mysqli php-json php php-devel
mariadb105-server
```

```
Last login: Thu Apr 16 20:50:40 2026 from 154.208.60.133
[ec2-user@ip-10-0-1-152 ~]$ sudo dnf update -y
Last metadata expiration check: 18:47:24 ago on Sun Apr 19 00:28:35 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-152 ~]$
```

```
[ec2-user@ip-10-0-1-152 ~]$ sudo dnf install -y httpd wget php-fpm php-mysqli php-json php php-devel mariadb105-server
Last metadata expiration check: 18:48:01 ago on Sun Apr 19 00:28:35 2026.
Package wget-1.21.3-1.amzn2023.0.4.x86_64 is already installed.
Dependencies resolved.
================================================================================================================
 Package                                  Architecture   Version                          Repository          Size
================================================================================================================
Installing:
 httpd                                    x86_64         2.4.66-1.amzn2023.0.1            amazonlinux         47 k
 mariadb105-server                        x86_64         3:10.5.29-1.amzn2023.0.1         amazonlinux         10 M
 php8.5                                   x86_64         8.5.4-1.amzn2023.0.1             amazonlinux         14 k
 php8.5-devel                             x86_64         8.5.4-1.amzn2023.0.1             amazonlinux        817 k
 php8.5-fpm                               x86_64         8.5.4-1.amzn2023.0.1             amazonlinux        3.0 M
 php8.5-mysqlnd                           x86_64         8.5.4-1.amzn2023.0.1             amazonlinux        154 k
Installing dependencies:
 annobin-docs                             noarch         12.69-1.amzn2023.0.1             amazonlinux         93 k
 annobin-plugin-gcc                       x86_64         12.69-1.amzn2023.0.1             amazonlinux        974 k
 apr                                      x86_64         1.7.5-1.amzn2023.0.4             amazonlinux        129 k
 apr-util                                 x86_64         1.6.3-1.amzn2023.0.2             amazonlinux         97 k
 apr-util-lmdb                            x86_64         1.6.3-1.amzn2023.0.2             amazonlinux         13 k
 autoconf                                 noarch         2.69-36.amzn2023.0.3             amazonlinux        666 k
 automake                                 noarch         1.16.5-9.amzn2023.0.3            amazonlinux        677 k
 cmake-filesystem                         x86_64         3.22.2-1.amzn2023.0.6            amazonlinux         12 k
 cpp                                      x86_64         11.5.0-5.amzn2023.0.5            amazonlinux         11 M
 emacs-filesystem                         noarch         1:28.2-3.amzn2023.0.10           amazonlinux        9.1 k
 gc                                       x86_64         8.0.4-5.amzn2023.0.2             amazonlinux        105 k
 gcc                                      x86_64         11.5.0-5.amzn2023.0.5            amazonlinux         36 M
 gcc-c++                                  x86_64         11.5.0-5.amzn2023.0.5            amazonlinux         15 M
```

Connect and Install the Web Stack

Start and enable the web server and database:

```
sudo systemctl start httpd

sudo systemctl enable httpd

sudo systemctl start mariadb

sudo systemctl enable mariadb
```

```
[ec2-user@ip-10-0-1-152 ~]$ sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl start mariadb
sudo systemctl enable mariadb
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
Created symlink /etc/systemd/system/mysql.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysqld.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[ec2-user@ip-10-0-1-152 ~]$
```

Create the Database

Log into the database as the root user:

```
sudo mysql -u root
```

create the database and a dedicated user:

```
CREATE DATABASE wordpress_db;

CREATE USER 'wp_user'@'localhost'
IDENTIFIED BY 'YourStrongPassword!';

GRANT ALL PRIVILEGES ON
wordpress_db.* TO
'wp_user'@'localhost';

FLUSH PRIVILEGES;

EXIT;
```

```
[ec2-user@ip-10-0-1-152 ~]$ sudo mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

```
MariaDB [(none)]> CREATE DATABASE wordpress_db;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]>
MariaDB [(none)]> CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'Trenz001!';
ERROR 1396 (HY000)NT ALL PRIVILEGE: Operation CREATE USER failed for 'wp_user'@'localhost'
MariaDB [(none)]>
MariaDB [(none)]> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]>
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]>
MariaDB [(none)]> EXIT;
Bye
[ec2-user@ip-10-0-1-152 ~]$
```

Download and Extract WordPress

Download the latest WordPress package:

wget https://wordpress.org/latest.tar.gz

Unzip the package:

tar -xzf latest.tar.gz

Copy all the extracted files to the Apache document root:

sudo cp -r wordpress/* /var/www/html/

```
[ec2-user@ip-10-0-1-152 ~]$ wget https://wordpress.org/latest.tar.gz
--2026-04-19 19:30:17--  https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 198.143.164.252, 2607:f978:5:8002::c68f:a4fc
Connecting to wordpress.org (wordpress.org)|198.143.164.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 27064862 (26M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz                   100%[===================================================>]  25.81M  91.8MB/s    in 0.3

2026-04-19 19:30:18 (91.8 MB/s) - 'latest.tar.gz' saved [27064862/27064862]

[ec2-user@ip-10-0-1-152 ~]$
```

```
[ec2-user@ip-10-0-1-152 ~]$ tar -xzf latest.tar.gz
[ec2-user@ip-10-0-1-152 ~]$
```

```
[ec2-user@ip-10-0-1-152 ~]$ sudo cp -r wordpress/* /var/www/html/
[ec2-user@ip-10-0-1-152 ~]$
```

Configure File Permissions

Change the ownership of the files to the Apache web user:

```
[ec2-user@ip-10-0-1-152 ~]$ sudo chown -R apache:apache /var/www/html
[ec2-user@ip-10-0-1-152 ~]$
```

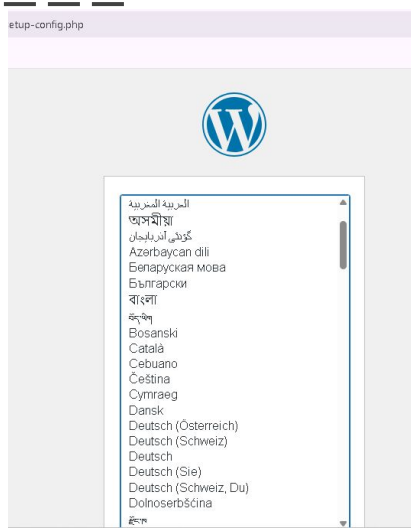
```
sudo chown -R apache:apache /var/www/html
```

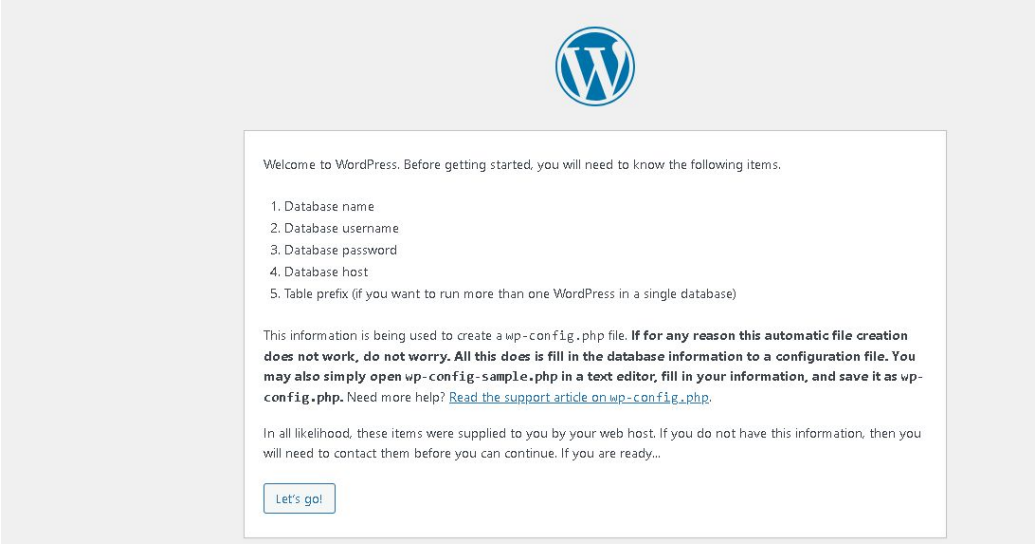
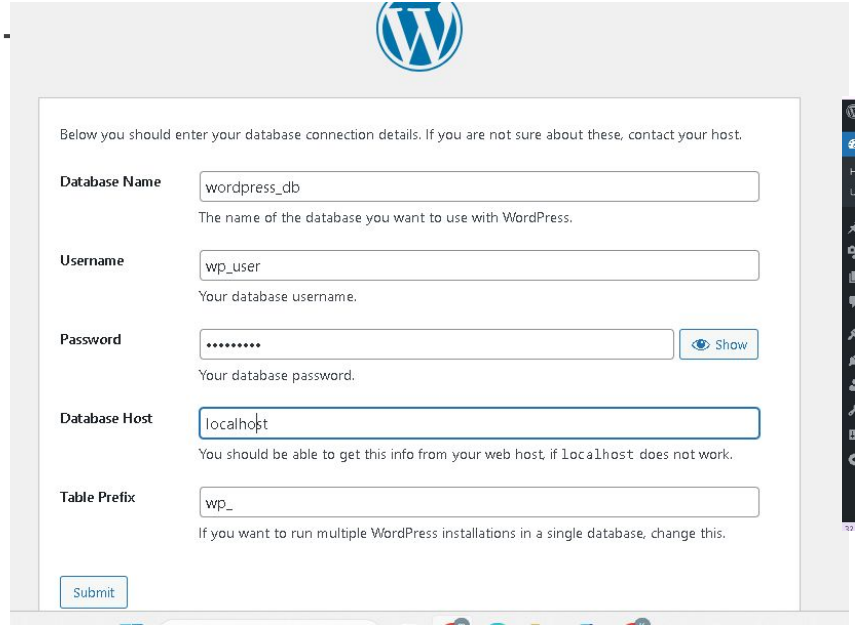
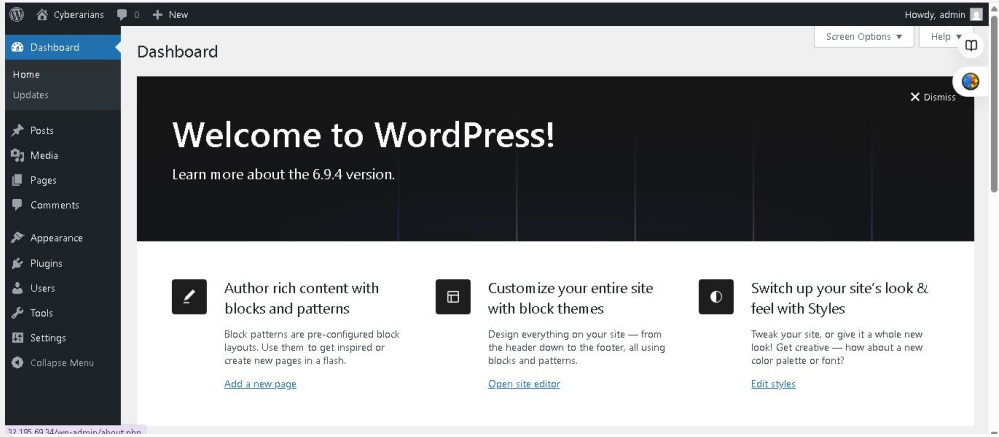
Set the correct directory permissions:

```
sudo chmod -R 755 /var/www/html
```

```
[ec2-user@ip-10-0-1-152 ~]$ sudo chmod -R 755 /var/www/html
[ec2-user@ip-10-0-1-152 ~]$
```

The Final Browser Setup





The Final Browser Setup